Estoque Fácil

Relatório técnico do sistema de controle de estoque

Projeto Integrador Transdisciplinar em Engenharia de Software II

Identificação	Informação
Aluno	Rafael Henrique da Silva
Curso	Engenharia de Software - Bacharelado
Instituição e polo	A preencher pelo aluno no template oficial
RGM	A preencher pelo aluno
Data e versão	01 de novembro de 2026 | versão 1.0
Natureza	Protótipo acadêmico local com cenário hipotético

Resumo

O Estoque Fácil organiza o cadastro de produtos e registra entradas e saídas em um pequeno comércio hipotético. A solução permite consultar saldos, identificar itens para reposição e preservar o histórico das movimentações. Foi implementada com Python, SQLite, HTML, CSS e JavaScript, sem pacotes externos para executar a aplicação.

Na verificação técnica desta versão foram executados 26 testes automatizados, com 0 falhas e 0 erros, além da inspeção funcional no navegador. A validação com cinco pessoas e a gravação narrada ainda não foram realizadas.

Condição de uso deste relatório

O trabalho foi iniciado do zero porque o projeto anterior não estava disponível. O enunciado da disciplina solicita continuidade do Projeto Integrador I; a aceitação desta nova base precisa ser confirmada com a tutoria. Este relatório é material de apoio e deve ser revisto e transposto para o template oficial antes da entrega.

Problema e planejamento

Cenário e objetivo

Em um cenário hipotético de pequeno comércio, registros manuais podem dificultar a consulta de saldo e o planejamento de reposições. O objetivo é construir uma aplicação local simples, que mantenha um catálogo consistente e impeça saídas incompatíveis com o estoque disponível. Não foram realizadas entrevistas ou pesquisas de campo para fundamentar este cenário.

Escopo da versão

A versão atende um operador em uma única máquina. Inclui cadastro e edição, movimentações, alerta de estoque mínimo, histórico, busca, filtros, exportação CSV e arquivamento com preservação de registros. Login, várias empresas, acesso em rede, integração fiscal e avaliação por custo médio não fazem parte desta versão.

Etapa	Entrega verificável
Planejamento	Cenário hipotético, requisitos, regras e casos de uso
Modelagem	Arquitetura, classes, sequência, atividade e banco relacional
Implementação	Apresentação web, serviço de regras e persistência SQLite
Verificação técnica	Testes unitários e de integração, inspeção no navegador
Validação e fechamento	Cinco opiniões reais, ajustes, vídeo e template oficial - pendentes

Critérios de aceite

Uma saída superior ao saldo deve produzir erro sem modificar o banco. Uma movimentação válida deve atualizar o saldo e gravar o histórico na mesma transação. Produtos com saldo igual ou inferior ao mínimo devem aparecer como reposição. Os dados devem continuar disponíveis após reabrir a aplicação.

Melhorias realizadas durante a construção

A revisão técnica incorporou controle de versão para impedir sobrescrita de edições antigas, transação imediata para serializar movimentações concorrentes, tratamento de códigos duplicados, proteção da exportação CSV e navegação inicial segura. São ajustes de engenharia e verificação automática, não avaliações atribuídas a colegas.

Requisitos e regras de negócio

ID	Requisito funcional	Evidência principal
RF01	Cadastrar produto com código único	test_create_and_initial_audit
RF02	Editar informações sem mudar saldo	test_update_preserves_balance
RF03	Registrar entrada e saída	test_entry_and_exit
RF04	Bloquear saída sem saldo	test_insufficient_stock_rolls_back
RF05	Consultar painel e reposição	test_summary_and_reopen_persistence
RF06	Buscar e filtrar produtos	Inspeção funcional da interface
RF07	Consultar histórico	test_read_repositories
RF08	Arquivar produto sem saldo	test_archive_requires_zero_and_preserves_history
RF09	Exportar catálogo em CSV	test_csv_utf8_and_formula_escape

Requisitos não funcionais

RNF01: persistir em SQLite sem servidor de banco separado. RNF02: executar localmente com Python 3.10 ou superior, sem instalação de pacotes de terceiros. RNF03: oferecer interface adaptável e operável por teclado. RNF04: validar entradas no servidor e usar consultas parametrizadas. RNF05: preservar a consistência de saldo e histórico. A inspeção visual não constitui certificação de acessibilidade.

Regras

RN01: código único e normalizado para letras maiúsculas, inclusive para produtos arquivados. RN02: quantidades inteiras entre zero e um milhão; movimentações exigem quantidade maior que zero. RN03: saldo nunca negativo. RN04: preço em centavos inteiros, não negativo e com até duas casas decimais. RN05: arquivamento apenas com saldo zero, sem apagar movimentações. RN06: preço atual altera o valor estimado do estoque, não recalcula custo médio. RN07: alterações de catálogo utilizam versão para detectar edição desatualizada.

Casos de uso e arquitetura



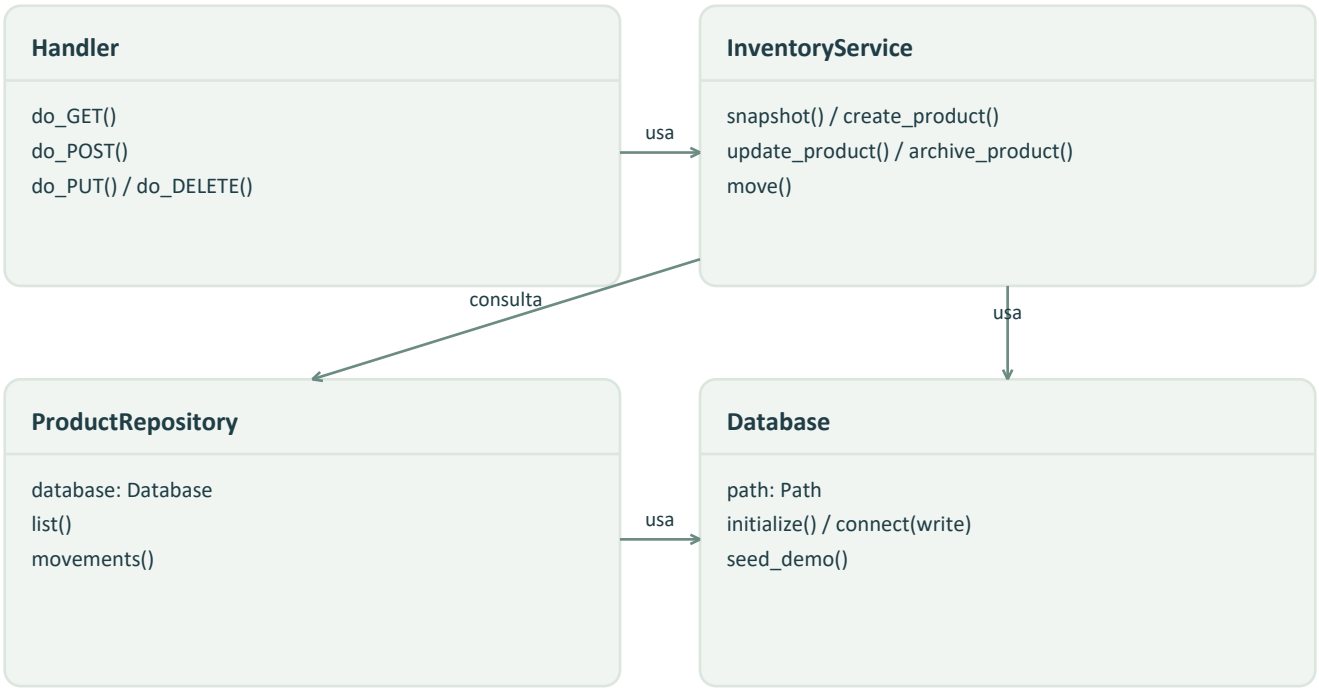
Organização MVC

A camada de apresentação contém as páginas e os eventos do navegador. O Handler da camada de controle interpreta a rota HTTP e produz respostas JSON. InventoryService concentra validação, saldo e transações. Database e ProductRepository encapsulam conexões e consultas. A interface não executa SQL.

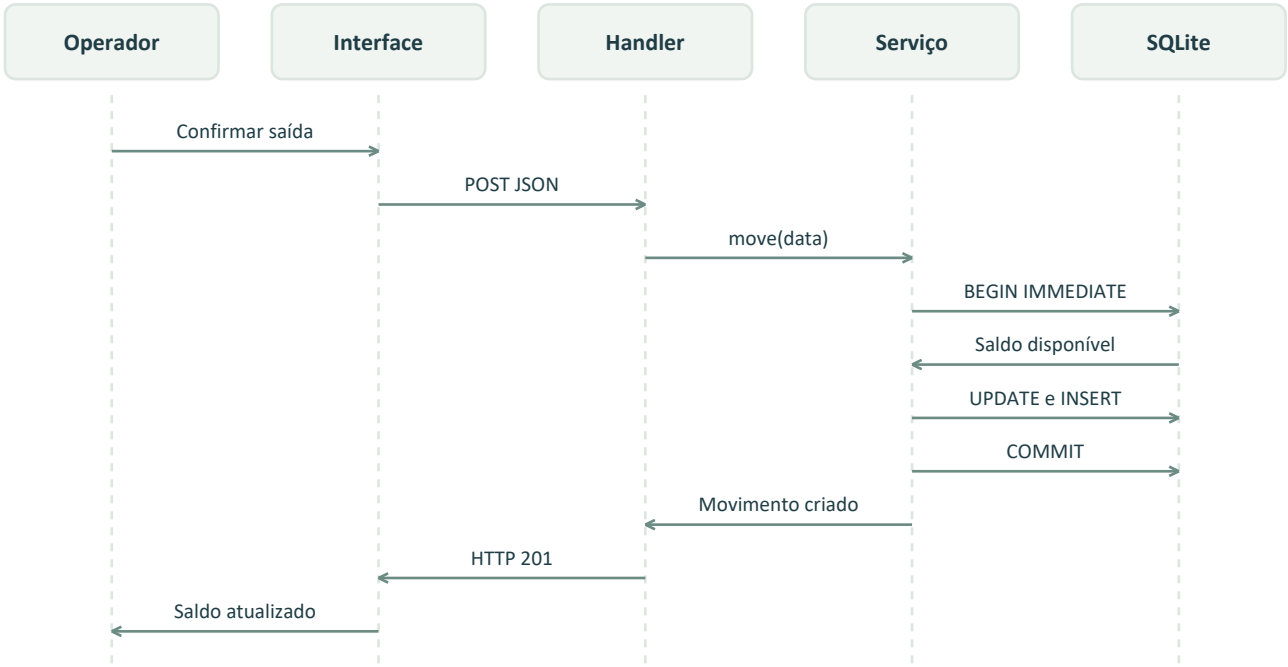


O processo é local e o servidor se vincula a 127.0.0.1. A API não oferece acesso externo ou autenticação. A apresentação recebe apenas os dados necessários ao fluxo da aplicação.

Modelagem de classes e sequência

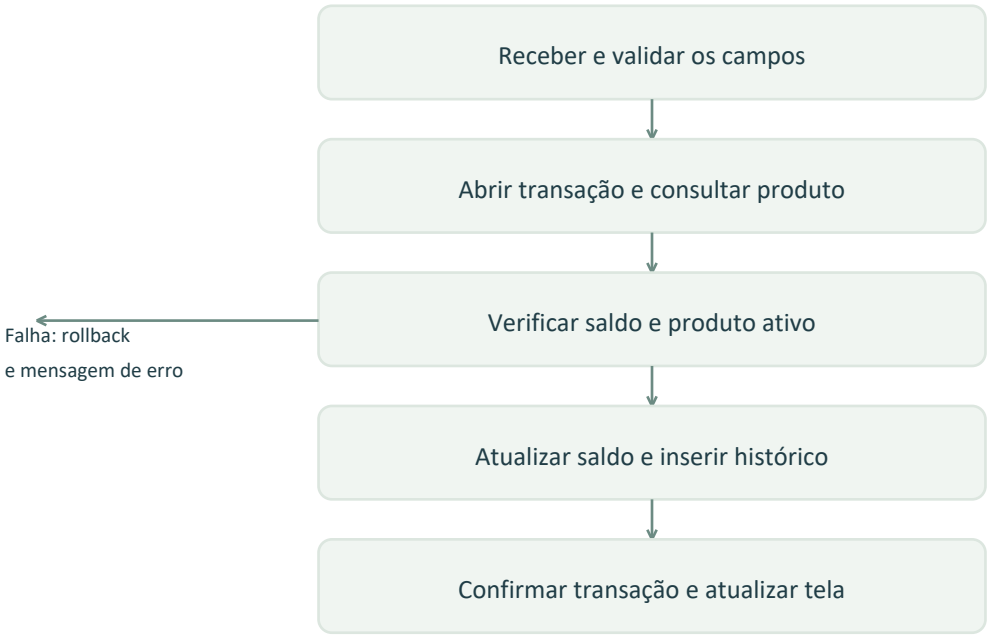


As dependências entre classes separam transporte HTTP, regras e persistência. Os testes instanciam o serviço com uma base temporária, sem depender da interface gráfica.



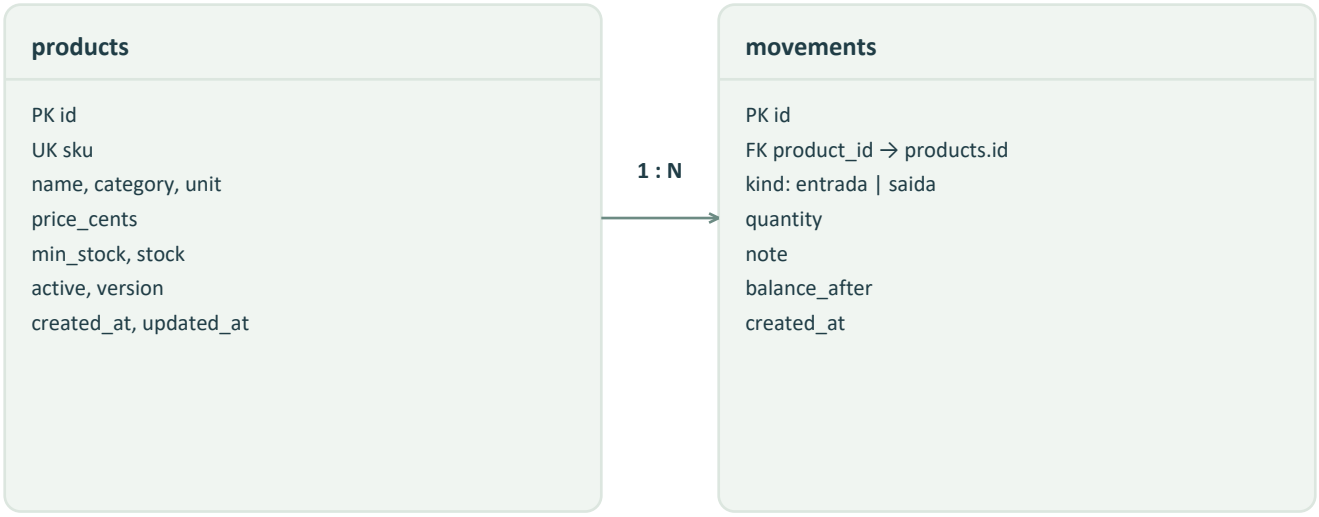
A sequência representa uma saída válida. Na alternativa de saldo insuficiente, InventoryService lança AppError, a transação é revertida e a API responde com HTTP 409; a interface mantém o formulário aberto e exibe a mensagem.

Fluxo de movimentação e dados



Projeto conceitual e lógico

Um produto possui zero ou muitas movimentações; cada movimentação pertence a exatamente um produto. A chave estrangeira product_id mantém o relacionamento. Código é chave candidata única. O arquivamento remove o produto do catálogo ativo, mas mantém o vínculo com as movimentações.



O saldo armazenado em products é uma desnormalização deliberada para consulta rápida. balance_after é uma fotografia do saldo após cada lançamento. Ambos são atualizados na mesma transação para evitar divergência. settings guarda apenas o marcador técnico da carga fictícia, sem relação com o domínio de estoque.

Dicionário de dados e API

Tabela e campo	Tipo	Finalidade
products.id / sku	INTEGER / TEXT	Chave primária e código único
products.name / category / unit	TEXT	Descrição, categoria e unidade un/cx/pct
products.price_cents	INTEGER	Preço unitário em centavos; mínimo zero
products.min_stock / stock	INTEGER	Limite de reposição e saldo atual
products.active / version	INTEGER	Estado ativo e versão de edição
products.created_at / updated_at	TEXT	Datas UTC em formato ISO 8601
movements.id / product_id	INTEGER	Chave primária e referência a products
movements.kind / quantity	TEXT / INTEGER	Entrada ou saída e quantidade positiva
movements.note	TEXT	Observação de até 250 caracteres
movements.balance_after	INTEGER	Saldo não negativo após a operação
movements.created_at	TEXT	Data UTC da movimentação
settings.key / value	TEXT	Controle técnico de carga de demonstração

Rotas implementadas

Método e rota	Comportamento
GET /api/state	Catálogo ativo, resumo e últimas 500 movimentações
POST /api/products	Cria produto e entrada inicial quando houver saldo
PUT /api/products/{id}	Edita informações com conferência de versão
DELETE /api/products/{id}	Arquiva produto com saldo zero
POST /api/movements	Valida, movimenta e preserva histórico
GET /api/export	Gera CSV do catálogo ativo

Respostas: 200 para consulta/edição; 201 para criação; 400 para dados inválidos; 403 para origem ou Host indevidos; 404 para item/rota inexistente; 409 para conflito; 413 para corpo excessivo; 415 para formato incorreto; 503 para banco temporariamente indisponível.

Manual de execução e operação

Executar

Extraia o ZIP completo. No Windows, abra INICIAR_DEMONSTRACAO.bat com Python 3.10 ou superior instalado. O endereço padrão é http://127.0.0.1:8765 e o terminal deve permanecer aberto. Para encerrar, pressione Ctrl+C. A demonstração usa seis produtos fictícios. INICIAR_ESTOQUE_VAZIO.bat usa outra base, inicialmente vazia.

Cadastrar e editar

Clique em Novo produto, informe nome, código, categoria, unidade, preço, saldo inicial e mínimo. O código aceita letras, números, hífen e sublinhado. O saldo inicial gera uma entrada. No catálogo, Editar altera informações do produto; para mudar quantidades, utilize Movimentar.

Movimentar

Escolha entrada ou saída, produto e quantidade positiva. A observação identifica o motivo. Confirme e confira o saldo. Se a saída ultrapassar o saldo, o formulário mostra o erro e a operação não é registrada. Se houver dúvida por interrupção de conexão, atualize e consulte o histórico antes de tentar novamente.

Consultar e organizar

O painel mostra produtos, quantidades totais, itens no mínimo ou abaixo dele e valor estimado. A soma de quantidades não converte caixas ou pacotes em unidades. Busque por nome, código ou categoria, com ou sem acentos. Use os filtros de reposição e tipo de movimento. Exporte o catálogo pelo botão CSV.

Arquivar e corrigir

Um produto só pode ser arquivado quando seu saldo é zero. O histórico continua visível e o código permanece reservado. Não há exclusão de movimentações pela interface; compense um lançamento incorreto com nova movimentação e descreva o motivo.

Persistência e backup

Os bancos ficam em data/demo.sqlite e data/estoque.sqlite. Para backup, encerre a aplicação e copie toda a pasta data. O pacote não inclui dados pessoais nem bancos já preenchidos. Não exponha o servidor por rede ou túnel: esta versão se destina à demonstração local.

Verificação técnica e resultados

A suíte foi executada em 2026-11-01T09:41:07-03:00, com Python 3.12.10. Resultado: 26 testes aprovados, nenhuma falha e nenhum erro. O registro completo está em EVIDENCIAS_TESTES.txt. Esses testes não substituem a validação por usuários.

Grupo	Casos verificados
Cadastro e edição	Saldo inicial auditado, zero inicial, unicidade, validação de preço, código e quantidade; edição preserva saldo; versão desatualizada bloqueada
Movimentação	Entrada, saída, saldo exato, saldo insuficiente, limite máximo, quantidade inválida e produto inexistente
Integridade	Rollback se o histórico falhar; duas saídas concorrentes não vendem além do saldo; persistência após nova conexão
Operação	Resumo, repositórios, carga fictícia idempotente e arquivamento com histórico preservado
HTTP e exportação	Fluxo completo pela API, JSON inválido, tipo/tamanho do corpo, Host/origem, navegação de documento, arquivos estáticos, rotas inexistentes e CSV

Inspeção funcional no Chrome

Em uma base exclusiva de QA, foi cadastrado o produto QA-001 com saldo 4. Uma entrada de 3 resultou em 7. A tentativa de saída 8 foi bloqueada. Uma saída válida de 5 resultou em 2, ativando o alerta de reposição. O nome foi editado e encontrado pela busca sem acentos. O arquivamento com saldo 2 foi bloqueado; após saída de 2, o produto foi arquivado e seus quatro lançamentos permaneceram no histórico.

A interface foi inspecionada na dimensão padrão do navegador e em largura de 390 pixels. A verificação confirma os fluxos exercitados neste ambiente; não garante compatibilidade com todos os navegadores, sistemas operacionais ou necessidades de acessibilidade.

Correção observada na revisão

O bloqueio inicial de requisições externas também impedia abrir a página a partir de outro site. A correção permite somente a navegação de documento para a raiz, mantendo bloqueadas chamadas externas à API. Foi incluído teste de regressão para essa distinção.

Evidências da interface

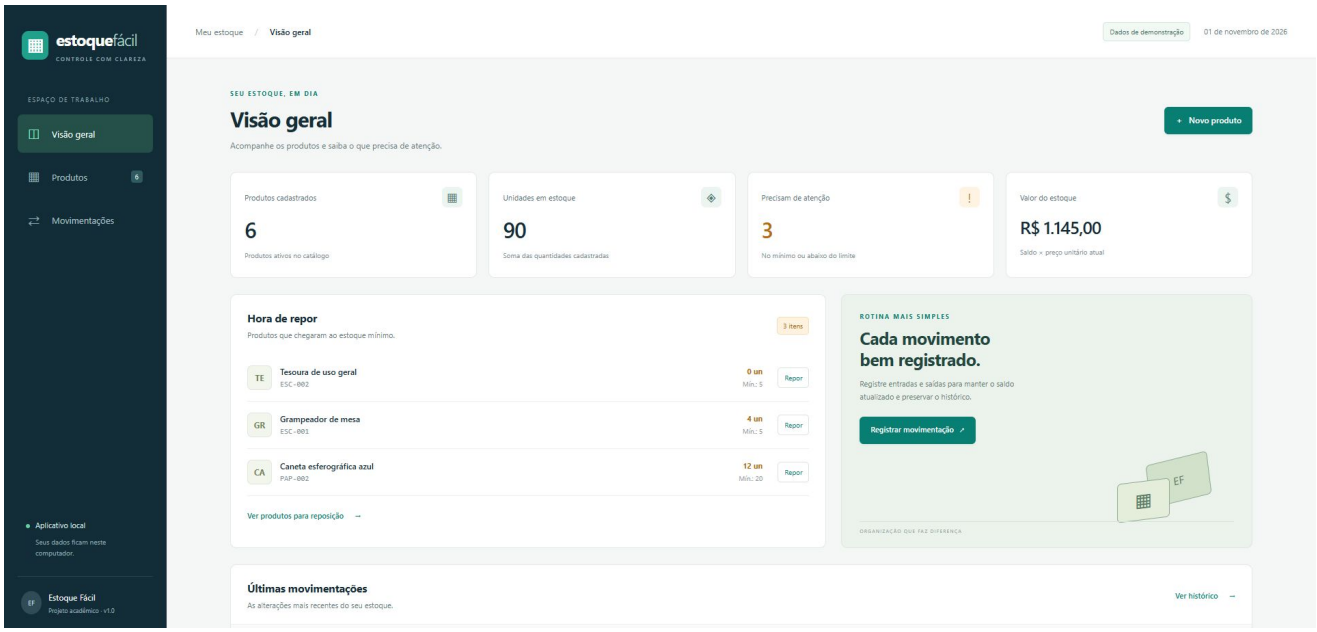


Figura 1. Painel real com a carga de demonstração fictícia. Mostra seis produtos, 90 quantidades cadastradas e três itens para reposição.

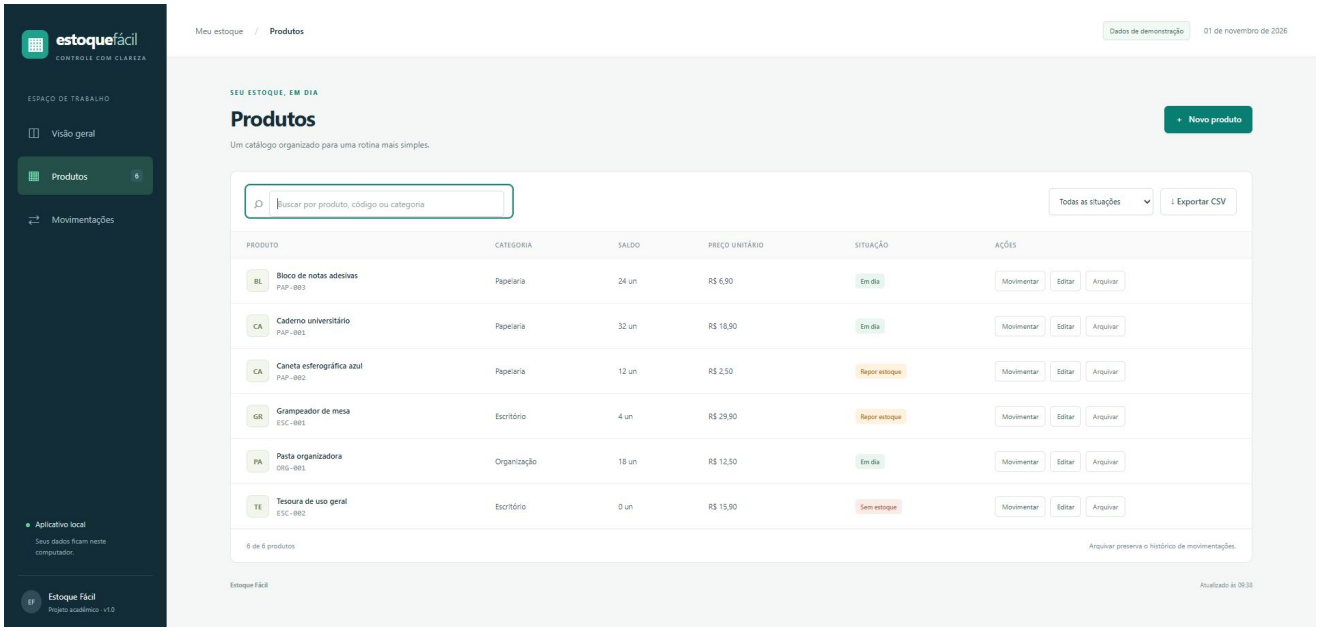


Figura 2. Catálogo real após os testes. O produto temporário de QA foi arquivado e os seis itens de demonstração permanecem ativos.

As capturas documentam a verificação técnica desta versão. Elas não representam atividade extensionista, implantação em um comércio real ou testes executados por colegas.

Evidências de erro e rastreabilidade

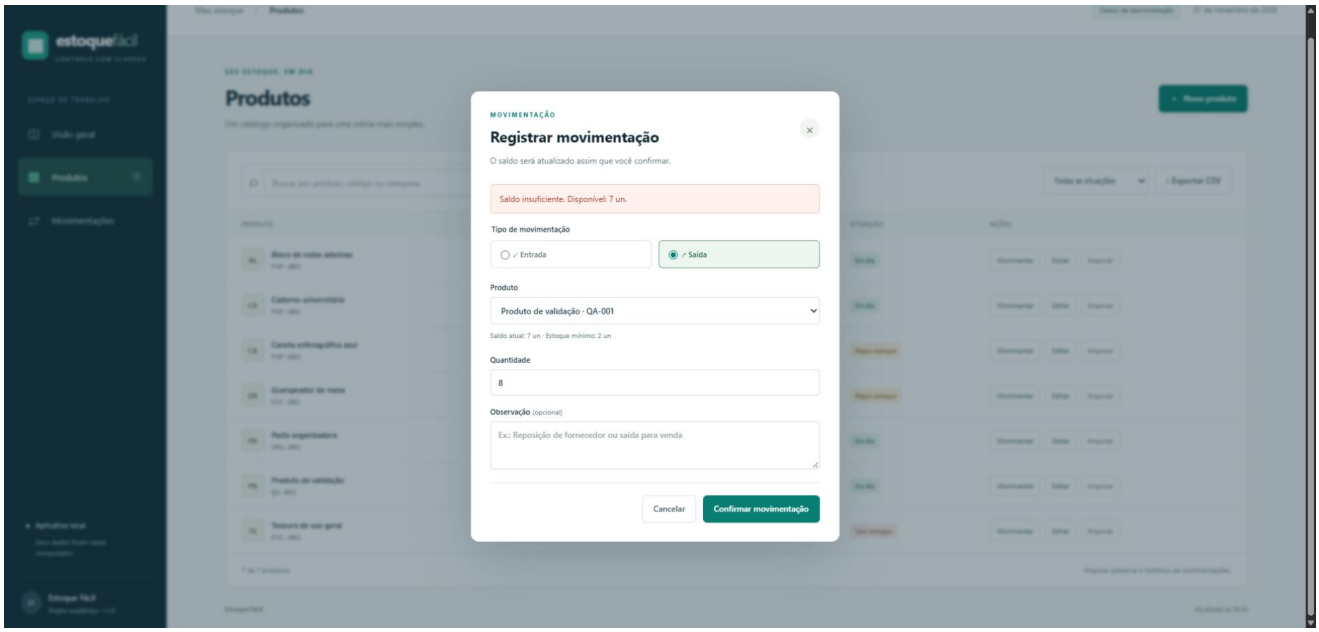


Figura 3. Mensagem de saldo insuficiente para saída de oito unidades quando o saldo era sete.

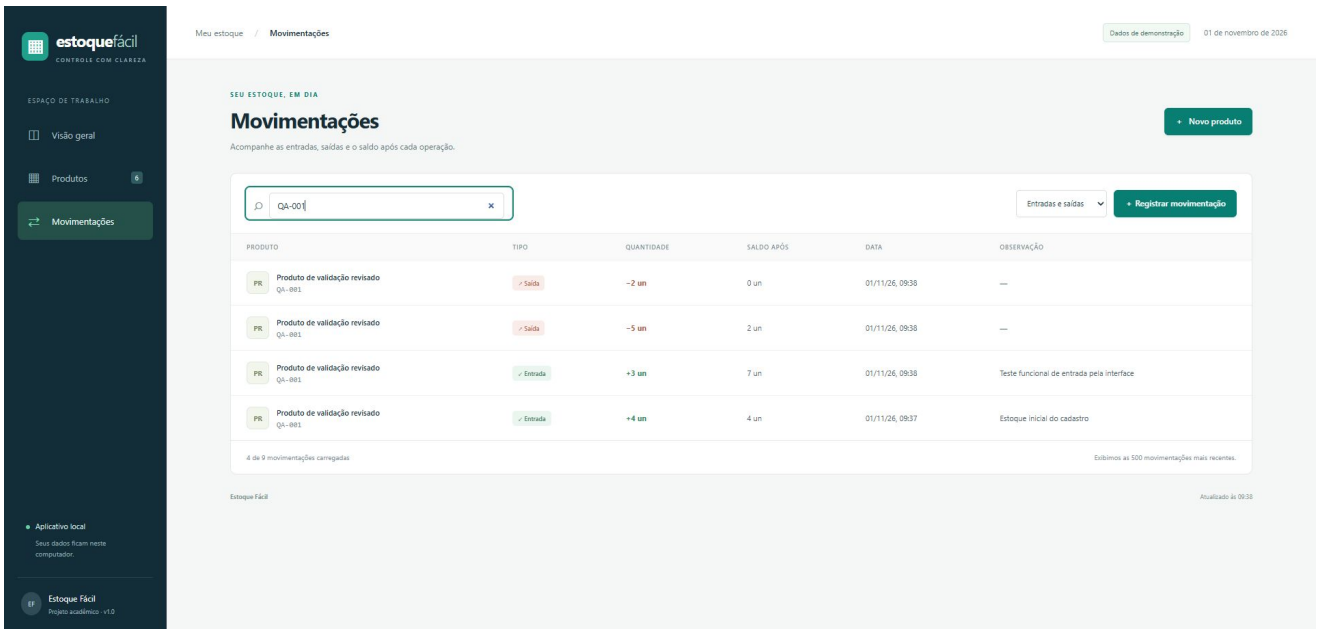


Figura 4. Histórico preservado do produto QA-001, com entrada inicial de quatro, entrada de três, saída de cinco e saída de dois.

Saldo final do produto de QA: zero. As tentativas recusadas não geraram movimentações. O nome exibido no histórico é o nome atual do produto, conforme a regra desta implementação.

Validação e preparação da entrega

Validação ainda não executada

A disciplina solicita cinco opiniões de colegas de curso ou profissionais. O pacote inclui VALIDACAO_USUARIOS.html, que pode ser aberto localmente. Cada pessoa deve testar o aplicativo e preencher nome, data, dispositivo, tarefas, funcionalidades que funcionaram, falhas, itens não testados e sugestões. O formulário salva JSON e permite impressão em PDF, com capturas opcionais. Nenhuma avaliação foi inventada.

Próximas etapas do aluno

Confirmar a aceitação do projeto iniciado do zero; revisar e compreender o código; preencher identificação; organizar as cinco avaliações reais; reunir os PDFs; registrar sugestões e ajustes em REGISTRO_MELHORIAS.csv; gravar a solução narrada; publicar o repositório remoto e transpor o conteúdo para o template oficial. O prazo observado no portal é 17/11/2026 às 23h59, horário de Brasília.

O enunciado diverge sobre a duração do vídeo: uma seção pede até cinco minutos, enquanto outra pede pelo menos cinco. O roteiro orienta aproximadamente cinco minutos, mas a duração deve ser confirmada com a tutoria.

Limitações e continuidade

A aplicação não tem autenticação, sincronização entre computadores, controle multiempresa, importação de dados ou cálculo fiscal. Evoluções possíveis: autenticação e implantação em servidor apropriado; paginação de histórico; restauração de itens arquivados; perfis e trilha de auditoria por operador. Essas funções não estão implementadas nesta versão.

Apoio de ferramentas

A implementação e esta documentação foram preparadas com apoio de um assistente de programação por IA. O aluno deve revisar o material, assumir apenas afirmações que possa demonstrar e observar as regras acadêmicas de autoria e uso de ferramentas de sua instituição.

Referências técnicas e arquivos

As referências abaixo são documentação primária das tecnologias utilizadas, consultadas em 01/11/2026. Os dados de demonstração, requisitos e resultados da implementação estão identificados neste relatório.

Tecnologias

Python Software Foundation. sqlite3 - DB-API 2.0 interface for SQLite databases. https://docs.python.org/3/library/sqlite3.html

Python Software Foundation. http.server - HTTP servers. https://docs.python.org/3/library/http.server.html. A documentação informa que o módulo não é recomendado para produção; a aplicação foi limitada ao uso local.

Python Software Foundation. unittest - Unit testing framework. https://docs.python.org/3/library/unittest.html

MDN Web Docs. The dialog element. https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/dialog

Material da disciplina

Cruzeiro do Sul Virtual. Projeto Integrador Transdisciplinar em Engenharia de Software II. Material didático: Situações-Problema 1, 2 e 3; Problema em Foco; Atividade de Entrega. Conteúdo consultado no ambiente autenticado do aluno.

Mapa do pacote

Arquivo ou pasta	Uso
README.md e arquivos BAT	Instruções e execução
estoque/ e static/	Código de modelo, serviço, controlador e apresentação
tests/	Suíte automatizada com bases temporárias
docs/schema.sql e diagramas.mmd	Esquema físico e diagramas editáveis
docs/imagens/	Capturas e diagramas da documentação
docs/VALIDACAO_USUARIOS.html	Avaliação real e impressão por pessoa
docs/ENTREGA.md e ROTEIRO_VIDEO.md	Pendências acadêmicas e apresentação